

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

Кафедра информационных систем и цифровых технологий

Работа допущена к защите

 Руководитель
«24» 12 2025 г.

КУРСОВОЙ ПРОЕКТ

по дисциплине «Проектирование информационных систем»

на тему: «Проектирование и разработка Android – приложения для выбора
фильмов для просмотра»

Студент Кор Королев А.С.

Шифр 220893

Институт приборостроения, автоматизации и информационных технологий

Направление подготовки 09.03.03 Прикладная информатика

Направленность (профиль) Интеллектуальная обработка данных

Группа 31ПИ

Руководитель Волков В.Н.

Оценка: « отлично »

Дата 24.12.2025

Орел 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

Кафедра информационных систем и цифровых технологий

УТВЕРЖДАЮ:

 и.о. зав. кафедрой

«23» сентября 2025 г.

ЗАДАНИЕ на курсовой проект

по дисциплине «Проектирование информационных систем»

Студент Королев А. С.

Шифр 220893

Институт приборостроения, автоматизации и информационных технологий

Направление подготовки 09.03.03 Прикладная информатика

Направленность (профиль) Интеллектуальная обработка данных

Группа 31ПИ

1 Тема курсового проекта

«Проектирование и разработка Android – приложения для выбора фильмов для просмотра»

2 Срок сдачи студентом законченной работы «24» декабря 2025

3 Исходные данные

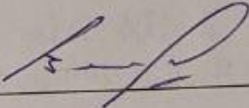
Методология проектирования информационных систем; базовые знания предметной области; основные положения проектирования и разработки Android - приложения.

4 Содержание курсового проекта

Описание предметной области;
Формализация процессов предметной области;
Проектирование системы;
Реализация программного продукта.

5 Отчетный материал курсового проекта

Пояснительная записка курсового проекта.

Руководитель _____  Волков В.Н.

Задание принял к исполнению: «23» сентября 2025

Подпись студента 

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ.....	6
2. ФОРМАЛИЗАЦИЯ ПРОЦЕССОВ ПРЕДМЕТНОЙ ОБЛАСТИ	9
2.1. Описание и анализ поставленной задачи	9
2.2. Функциональные требования	9
2.3. Нефункциональные требования	10
2.4. Обзор аналогов	10
2.5. Построение модели процессов предметной области	13
3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ.....	16
3.1. Архитектура.....	16
3.2. Проектирование системы хранения и обработки данных	20
3.3. Проектирование объектной модели приложения.....	24
3.4. Проектирование логики диалога с пользователем	28
4. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА.....	33
ЗАКЛЮЧЕНИЕ	38
СПИСОК ЛИТЕРАТУРЫ.....	40
ПРИЛОЖЕНИЕ А	
ПРИЛОЖЕНИЕ Б	

ВВЕДЕНИЕ

В современном мире кинематограф предлагает огромное количество фильмов и сериалов различных жанров, эпох и направлений. Этот избыток контента, доступного на множестве стриминговых платформ, приводит к возникновению «парадокса выбора»: пользователи тратят значительное время на поиск и выбор произведения для просмотра, что зачастую снижает удовольствие от самого процесса. В результате возникает потребность в интеллектуальном помощнике, способном не только агрегировать информацию из различных источников, но и персонализировать рекомендации на основе предпочтений, контекста и настроения пользователя. Разработка Android-приложения для решения этой задачи является актуальной, так как смартфоны остаются самым доступным и распространенным устройством для быстрого и удобного взаимодействия с сервисами.

Цель проекта – разработать прототип Android-приложения для помощи пользователям в выборе фильмов, сочетающего в себе функции агрегации информации, рекомендательной системы и социального взаимодействия. Для достижения цели курсового проекта необходимо выполнить следующие задачи:

1. Проанализировать и описать предметную область;
2. Формализовать процессы взаимодействия пользователя с системой;
3. Спроектировать продукт с учетом функциональных требований;
4. Реализовать и протестировать прототип программного продукта.

1. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

Кино, как один из ключевых видов современного искусства и развлечения, обладает мощной силой эмоционального воздействия, способностью к образованию и формированию культурного контекста. Однако стремительный рост объемов производимого и дистрибутируемого контента привел к парадоксальной ситуации: при кажущейся неограниченной доступности фильмов и сериалов, пользователь тратит непропорционально большое количество времени и усилий на сам процесс выбора, что зачастую снижает удовлетворенность от конечного просмотра. Эта проблема усугубляется фрагментацией контента по множеству стриминговых платформ, каждая из которых предлагает собственную, часто ограниченную, рекомендательную систему, нацеленную в первую очередь на удержание внутри экосистемы.

Регулярный и осознанный выбор контента для просмотра помогает не только оптимизировать досуг, но и глубже понять собственные эстетические предпочтения, открыть новые жанры и культурные явления. Благодаря этому пользователь может выстраивать более осмысленную и удовлетворяющую его медиа-диету, избегая как информационной перегрузки, так и попадания в «пузырь» однотипного контента, навязанного коммерческими алгоритмами.

Ведение истории просмотров и оценок полезно не только для формирования персональных рекомендаций, но и для выявления скрытых паттернов. Например, пользователь может обнаружить, что его привлекают не просто фильмы определенного жанра, а работы конкретных операторов, сценаристов или нарративные структуры, объединяющие произведения из разных стран и эпох. На основе собранных данных можно целенаправленно исследовать кинематограф, расширяя кругозор и формируя индивидуальную киноколлекцию. Кроме того, фиксация впечатлений помогает отслеживать эволюцию

собственного вкуса и делиться осмысленными рекомендациями с единомышленниками.

Современные мобильные технологии предоставляют уникальные возможности для создания персонализированного гида в мире кино. Операционная система Android, обладающая широкой распространенностью и открытыми инструментами разработки, является идеальной платформой для такого приложения. Android-приложения могут предлагать глубокую интеграцию с устройством, интуитивно понятный интерфейс и возможность работать с большими объемами данных в режиме реального времени, обеспечивая мгновенный отклик на действия пользователя.

Целью данного проекта является проектирование и разработка Android-приложения, которое будет помогать пользователям открывать, выбирать и систематизировать фильмы для просмотра. На этапе проектирования предусмотрено, что приложение будет выполнять ряд функций, направленных на создание удобного инструмента для навигации по киноконтенту:

- Лента рекомендаций. Приложение на старте предлагает пользователю фильмы и сериалы для просмотра. Алгоритм будет комбинировать методы коллаборативной фильтрации (на основе схожести с другими пользователями).

- Управление персональной кинотекой. Пользователь сможет создавать собственную коллекцию. («Избранное»). Эти данные планируется хранить локально на устройстве.

Для реализации функции агрегации актуальных данных в разрабатываемом приложении на этапе проектирования предусмотрено использование стороннего API сервиса Кинопоиск (Kinipoisk API). Этот инструмент позволит организовать эффективный процесс получения и обновления метаданных, необходимых для работы приложения. Через API будет загружаться информация о фильмах и сериалах: постеры, описания, рейтинги, трейлеры. Такой подход

обеспечит высокую точность, централизованность данных и избавит от необходимости самостоятельного поддержания огромной базы.

Таким образом, разрабатываемое Android-приложение является комплексным инструментом, объединяющим функции агрегатора информации, персонального кинокритика и интеллектуального рекомендательного сервиса. Внедрение системы работы с внешним API, оптимизированное локальное хранение данных и алгоритмы рекомендаций обеспечивают современность, надежность и функциональность проекта. Реализация проекта позволит не только упростить процесс выбора фильма, но и превратить его в увлекательное и познавательное путешествие по миру кинематографа, что делает данное приложение актуальным и значимым решением в контексте современных медиа-привычек.

2. ФОРМАЛИЗАЦИЯ ПРОЦЕССОВ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Описание и анализ поставленной задачи

В результате выполнения курсового проекта должно быть разработано Android-приложение для работы с каталогом фильмов, предназначенное для просмотра информации о фильмах, трейлерах и рецензиях, а также для формирования списка избранных фильмов пользователем.

Разрабатываемое приложение должно обеспечивать следующие основные возможности:

1. Отображение списка фильмов, получаемого с удалённого сервера с использованием Kinopoisk API;
2. Поддержку бесконечной прокрутки списка фильмов (пагинации) для удобного просмотра большого объёма данных;
3. Предоставление пользователю возможности выбора конкретного фильма из списка;
4. Отображение детальной информации о выбранном фильме, включая:
 - название фильма;
 - год выпуска;
 - постер;
 - описание (аннотацию);
 - список рецензий;
 - список трейлеров;
5. Предоставление пользователю возможности добавления фильма в список избранных;
6. Предоставление пользователю возможности удаления фильма из списка избранных;

7. Отображение списка избранных фильмов, сохранённых в локальной базе данных;

8. Обеспечение доступа к избранным фильмам при отсутствии интернет-соединения.

2.2 Функциональные требования

Приложение должно удовлетворять следующим функциональным требованиям:

- При запуске приложения выполнять проверку доступности интернет-соединения;
- При наличии интернет-соединения загружать список фильмов с удалённого сервера и отображать его пользователю;
- При отсутствии интернет-соединения отображать пустой список фильмов с возможностью перехода в раздел избранных;
- Обеспечивать корректную обработку ошибок сети и ошибок получения данных от сервера;
- При выборе фильма из списка загружать и отображать детальную информацию о фильме, включая рецензии и трейлеры;
- Обеспечивать асинхронную загрузку данных для предотвращения блокировки пользовательского интерфейса;
- Предоставлять пользователю возможность добавления и удаления фильма из списка избранных с сохранением изменений в локальной базе данных;
- Обеспечивать хранение избранных фильмов в локальной базе данных с использованием технологии Room;
- Обеспечивать корректное отображение списка избранных фильмов при повторном запуске приложения;

- Предоставлять пользователю возможность перехода между основным экраном, экраном деталей фильма и экраном избранных фильмов.

2.3 Нефункциональные требования

К разрабатываемому приложению предъявляются следующие нефункциональные требования:

- приложение должно иметь интуитивно понятный и удобный пользовательский интерфейс;
- время отклика пользовательского интерфейса при выполнении основных операций не должно превышать 1–2 секунд при наличии интернет-соединения;
- приложение должно корректно работать на устройствах под управлением операционной системы Android;
- архитектура приложения должна обеспечивать разделение ответственности между слоями пользовательского интерфейса, бизнес-логики и работы с данными;
- приложение должно быть устойчивым к ошибкам сети и некорректным данным, получаемым от сервера;

2.4 Обзор аналогов

В контексте современного цифрового медиапотребления, где поиск и выбор контента становятся все более сложной задачей из-за его изобилия и фрагментации, существует множество решений, направленных на помощь пользователям в навигации по миру кино. Эти аналоги предлагают различные подходы к решению одной общей задачи – эффективному поиску, оценке и выбору фильмов для просмотра.

- 1) Кинопоиск – это ведущий русскоязычный онлайн-сервис о кино, сочетающий в себе базу данных, агрегатор рецензий и социальную платформу. Сервис предлагает пользователям обширную информацию

о фильмах и сериалах, включая рейтинги, отзывы зрителей и критиков, подробные статьи и новости индустрии. Кинопоиск ориентирован на создание комплексной экосистемы для киноэнтузиастов, где ключевую роль играют пользовательский контент (рецензии, списки) и авторитетное редакционное мнение. Его сила – в глубокой локализации, активном сообществе и статусе основного источника информации о кино в рунете.

2) IMDb (Internet Movie Database) – это крупнейшая в мире международная база данных о кинематографе, телевидении и видеоиграх. Сервис предоставляет исчерпывающие профессиональные метаданные (актерский состав, съемочная группа, синопсисы, trivia), а также пользовательские рейтинги и обзоры. IMDb функционирует как универсальный справочник и ориентирован на глобальную аудиторию. Его ключевыми преимуществами являются беспрецедентный охват, историческая глубина данных и интеграция с сервисом стриминга Prime Video. Основное внимание уделяется объективной информативности и авторитетности как главного архива киноиндустрии.

Для лучшей структурированности анализа и визуализации информации составим таблицу аналогов, которая отразит ключевые функциональные особенности и характеристики существующих решений (Таблица 1). Таблица позволит увидеть, насколько каждый из выбранных аналогов соответствует выдвинутым требованиям проекта. Для сравнения выбраны два популярных сервиса, которые частично или полностью реализуют схожие функциональные возможности, направленные на выбор и ознакомление с фильмами и сериалами.

Таблица 1 – Анализ аналогов с учётом ключевых требований

Критерии / Аналоги	Название	
	Кинопоиск	IMDb
Избранное	+	+

Локальное хранение избранных фильмов	—	—
Трейлеры	+	+
Рецензии	+	+
Алгоритмы рекомендаций	+	+
Наличие платных функций	+	+
Удобство интерфейса (по отзывам пользователей)	+—	+—

Анализ существующих решений помогает определить, какие функции следует включить в проектируемое приложение, чтобы сделать его максимально удобным и полезным для пользователей. Также он позволяет понять, какие недостатки текущих сервисов можно учесть и исправить, предложив более продуманное и уникальное решение.

2.5 Построение модели процессов предметной области

Для построения концептуальной модели предметной области разработана диаграмма вариантов использования (use-case), представленная на Рисунке 1. Данная диаграмма отображает функциональные возможности системы, определяет границы и контекст её работы, а также формулирует общие требования к функциональному поведению проектируемой системы.

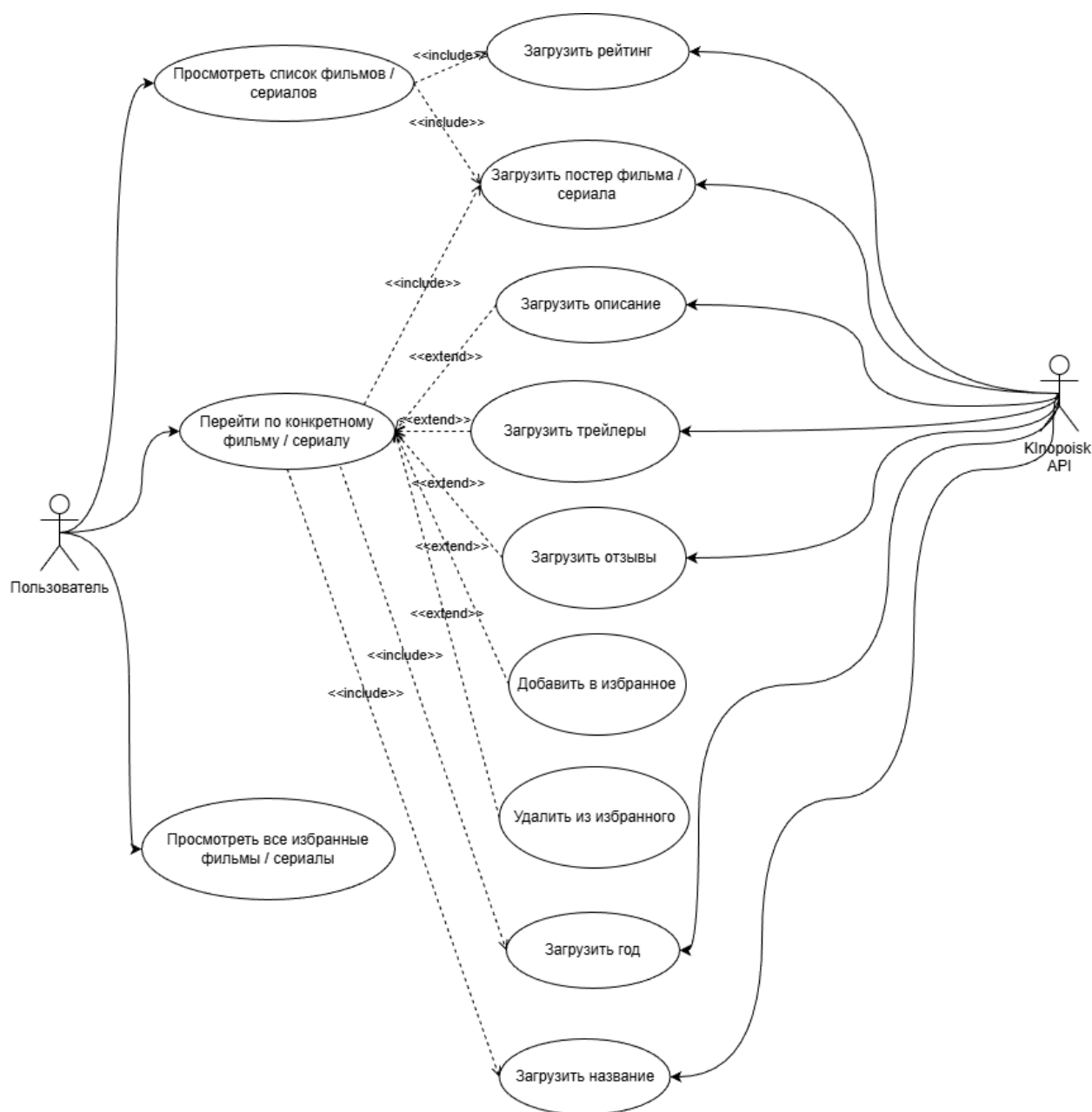


Рисунок 1 – Диаграмма вариантов использования.

Диаграмма отражает функциональные возможности приложения для взаимодействия пользователя с каталогом фильмов и сериалов. В системе выделены два ключевых типа сущностей: активный участник — Пользователь, и пассивная внешняя система — КиноПоиск API. Основные варианты использования описывают действия пользователя по поиску, просмотру информации и управлению личным списком избранного контента.

1. Пользователь

Пользователь является центральным актором системы, выполняющим основные действия по работе с каталогом. Его варианты использования можно разделить на три логические группы.

А. Просмотр каталога и поиск информации

- Это базовая и самая объемная группа функций, обеспечивающая навигацию по контенту.
- «Просмотреть список фильмов / сериалов»: Это начальное и центральное действие пользователя. Оно включает в себя (отношение «include») три обязательных подпроцесса для отображения корректной информации в списке:
 - «Загрузить рейтинг»: Получение текущего рейтинга для каждого элемента списка.
 - «Загрузить постер фильма / сериала»: Получение обложки для визуального отображения.
 - «Загрузить описание»: Получение краткого синописа.
- «Перейти по конкретному фильму / сериалу»: Это действие расширяет («extend») базовый просмотр списка. Оно является опциональным и выполняется, когда пользователь решает детально изучить какой-либо конкретный title из общего списка.
- «Загрузить год»: Это обязательное действие («include»), выполняемое в рамках перехода к конкретному фильму/сериалу для отображения года выпуска.

В. Управление личным списком «Избранное»

- Эта группа функций позволяет пользователю персонализировать свой опыт.
- «Добавить в избранное»: Пользователь может сохранить понравившийся фильм или сериал в свой персональный список. Это действие является расширением («extend») варианта использования

«Перейти по конкретному фильму / сериалу», так как обычно выполняется уже на странице деталей.

- «Просмотреть все избранные фильмы / сериалы»: Позволяет получить доступ к сохраненному персональному списку.
- «Удалить из избранного»: Обеспечивает управление содержимым списка «Избранное».

С. Получение расширенной информации

- Эти действия предоставляют пользователю дополнительный медиа-контент и мнения.
- «Загрузить трейлеры»: Опциональное действие («extend»), которое пользователь может инициировать на странице конкретного фильма/сериала для просмотра видео-трейлеров.
- «Загрузить отзывы»: Еще одно опциональное расширение («extend») базового просмотра деталей, позволяющее ознакомиться с отзывами других зрителей.

2. Кинопоиск API

Данная представляет собой внешнюю службу, с которой взаимодействует само приложение. Все действия пользователя, связанные с загрузкой данных (рейтинга, описания, постеров, трейлеров, отзывов), в конечном итоге выполняются путем обращения приложения к Кинопоиск API. Таким образом, API выступает в роли единого и обязательного источника данных для реализации всех перечисленных вариантов использования.

На диаграмме присутствуют как ассоциации, так и отношения включения. Отношения включения используются для детализации функциональных процессов, таких как создание сообщения или настройка взаимодействия с пользователем.

3. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

3.1. Архитектура

Для реализации Android-приложения в рамках курсового проекта была выбрана клиент-серверная архитектура с использованием паттерна MVVM (Model–View–ViewModel). Данный подход позволяет разделить ответственность между пользовательским интерфейсом, логикой обработки данных и источниками данных, обеспечивая удобство использования приложения, устойчивость к ошибкам и возможность дальнейшего расширения функциональности.

В рамках выбранной архитектуры мобильное приложение выступает в роли клиента, а удалённый сервер фильмов — в роли сервера, предоставляющего данные о фильмах, трейлерах и рецензиях через Kinopoisk API. Взаимодействие между клиентом и сервером осуществляется посредством HTTP-запросов с использованием интерфейса ApiService.

В процессе проектирования архитектуры приложения были рассмотрены следующие подходы к организации работы с данными:

- использование только удалённого сервера без локального хранения данных;
- использование удалённого сервера в сочетании с локальной базой данных;
- полностью автономная работа приложения с локальным хранилищем.

С учётом функциональных требований, а также необходимости обеспечения доступа к избранным фильмам при отсутствии интернет-соединения, наиболее подходящим вариантом оказался второй подход, сочетающий работу с удалённым сервером и локальной базой данных. Такой подход позволяет обеспечить актуальность данных, получаемых из сети, и одновременно сохранять пользовательские данные локально.

В рамках реализации приложения удалённый сервер используется для получения основной информации о фильмах, включая список фильмов, их описание, трейлеры и рецензии. Локальная база данных, реализованная с использованием технологии Room, применяется для хранения списка избранных фильмов пользователя. Это позволяет обеспечить доступ к сохранённым данным даже при отсутствии интернет-соединения.

Использование клиент-серверной архитектуры имеет ряд преимуществ. Во-первых, обеспечивается разделение логики приложения и источников данных, что упрощает сопровождение и тестирование системы. Во-вторых, обновление данных на сервере не требует внесения изменений в клиентскую часть приложения. В-третьих, данный подход позволяет масштабировать серверную часть без изменения архитектуры мобильного приложения.

Однако выбранная архитектура также имеет определённые ограничения. Работа приложения напрямую зависит от доступности интернет-соединения при загрузке данных с сервера. Кроме того, при увеличении объёма данных или количества пользователей возможны ограничения, связанные с производительностью сервера и лимитами API. Эти ограничения могут быть частично нивелированы за счёт оптимизации запросов и использования механизмов кэширования.

Тем не менее, выбранный подход является оптимальным для текущего этапа разработки, так как он позволяет сосредоточиться на реализации функциональности приложения без необходимости развёртывания и администрирования собственной серверной инфраструктуры. Использование готового Kinopoisk API существенно снижает временные затраты на разработку и упрощает процесс тестирования.

В перспективе архитектура приложения может быть расширена за счёт внедрения дополнительного серверного слоя, например, собственного backend-сервиса, который будет выполнять агрегацию данных, хранение пользовательских настроек или реализацию рекомендационных алгоритмов. Также

возможна интеграция с облачными платформами, такими как Google Firebase или AWS, для обеспечения масштабируемости и повышения отказоустойчивости системы.

Таким образом, выбранная архитектура является сбалансированным решением, обеспечивающим достаточную функциональность, удобство разработки и возможность дальнейшего развития приложения.

Для оценки и выбора оптимальной архитектуры был произведён сравнительный анализ различных подходов. Основные характеристики были сведены в сравнительную таблицу. (Таблица 2).

Таблица 2 – Сравнение архитектур

Критерий	Только сервер	Сервер + локальная БД	Только локальная БД
Преимущества	Актуальные данные	Баланс актуальности и офлайн-доступа	Полная автономность
Недостатки	Нет офлайн-доступа	Усложнение архитектуры	Отсутствие обновлений
Скорость внедрения	Высокая	Средняя	Высокая
Стоимость	Высокая	Средняя (зависит от сервиса).	Минимальная
Ограничения	Зависимость от сети	Ограничения API	Ограниченный функционал
Перспективы масштабирования	Высокие	Высокие	Низкие

Анализ показывает, что для текущего этапа проекта наиболее подходящим является использование архитектуры, сочетающей работу с удалённым сервером и локальной базой данных. Данный подход обеспечивает

компромисс между функциональностью, удобством использования и возможностью масштабирования.

На Рисунке 2 представлена упрощённая схема клиент-серверной архитектуры Android-приложения для работы с фильмами. Пользователь взаимодействует с мобильным приложением через пользовательский интерфейс. Действия пользователя обрабатываются слоем View, после чего передаются в ViewModel, отвечающую за бизнес-логику приложения.

ViewModel взаимодействует с удалённым сервером фильмов через REST API с использованием интерфейса ApiService, а также с локальной базой данных через MovieDao и MovieDatabase. Полученные данные обрабатываются и передаются обратно в пользовательский интерфейс для отображения.

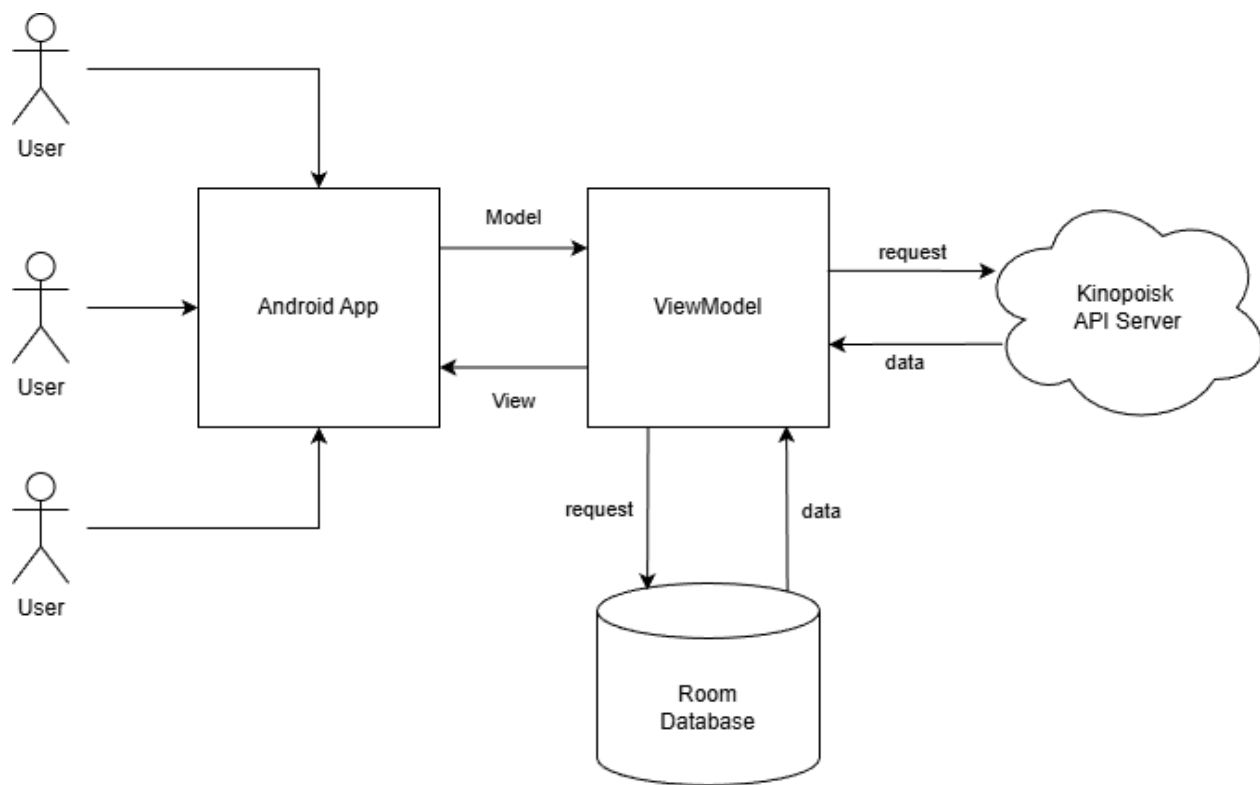


Рисунок 2 – Упрощённая схема клиент-серверной архитектуры.

Схема клиент-серверной архитектуры, представленная на Рисунке 4, отражает основные этапы обработки запросов, начиная с инициализации запроса от пользователя и заканчивая возвращением результата.

Основные этапы работы системы:

- пользователь инициирует действие в мобильном приложении (запуск приложения, выбор фильма, добавление в избранное);
- пользовательский интерфейс передаёт событие в соответствующую ViewModel;
- ViewModel формирует запрос к удалённому серверу через Kinopoisk API или обращается к локальной базе данных;
- сервер возвращает данные о фильмах, трейлерах и рецензиях;
- полученные данные преобразуются в модельные объекты и отображаются пользователю;
- при работе с избранными фильмами данные сохраняются или извлекаются из локальной базы данных.

Такая архитектура обеспечивает логическое разделение компонентов приложения, устойчивость к ошибкам сети, удобство сопровождения и возможность масштабирования. Использование паттерна MVVM позволяет минимизировать связность компонентов и повысить читаемость кода, что особенно важно при разработке мобильных приложений среднего уровня сложности.

3.2. Проектирование системы хранения и обработки данных

В рамках курсового проекта для хранения и обработки данных в Android-приложении используется локальная база данных, реализованная с применением библиотеки Room, входящей в состав Android Jetpack. Данное решение обеспечивает удобный, безопасный и эффективный способ работы с данными непосредственно на устройстве пользователя.

Система хранения данных предназначена для сохранения информации об избранных фильмах пользователя и обеспечения доступа к этим данным при отсутствии интернет-соединения. Основные данные о фильмах, такие как список фильмов, трейлеры и рецензии, загружаются с удалённого сервера

через Kinopoisk API и не сохраняются полностью в локальной базе данных, что позволяет избежать избыточного хранения информации.

Выбор технологии хранения данных

Для реализации локального хранилища была выбрана библиотека Room, представляющая собой объектно-реляционную обёртку над SQLite. Данный выбор обусловлен следующими причинами:

- Room обеспечивает строгую типизацию данных и проверку SQL-запросов в базе данных на этапе компиляции нашего Android-приложения
- библиотека упрощает работу с базой данных за счёт использования аннотаций и различных DAO-интерфейсов, которые применяются в проекте;
- Room является рекомендуемым решением для многих Android-приложений и полностью и успешно интегрируется с архитектурным паттерном MVVM;
- использование локальной базы данных позволяет обеспечить офлайн-доступ к сохранённым фильмам.

Использование внешних серверных СУБД, таких как MySQL или PostgreSQL, в рамках данного проекта является нецелесообразным, так как объём хранимых данных невелик, а необходимость в централизованном хранении пользовательской информации отсутствует.

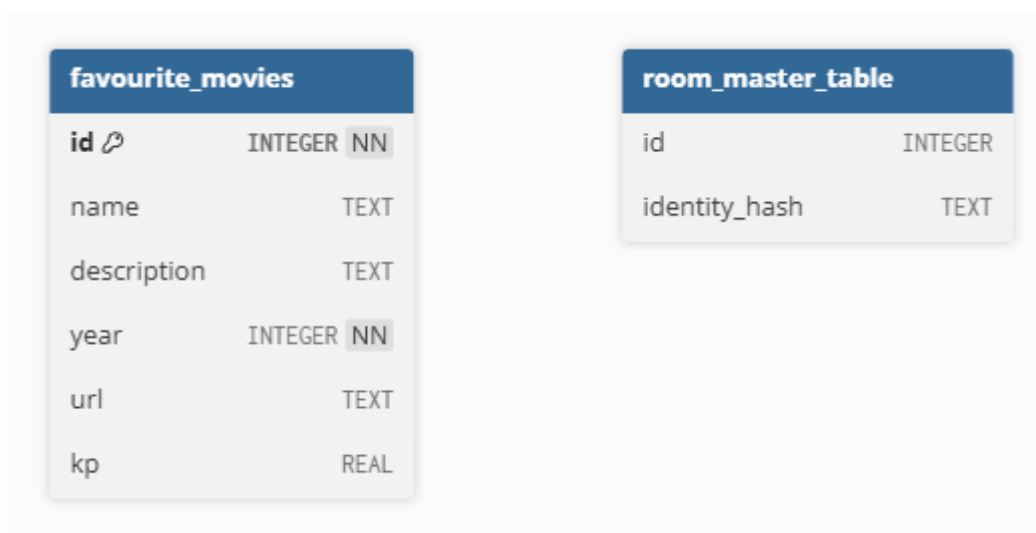


Рисунок 3 – Предполагаемая физическая схема базы данных для Room.

Структура базы данных

Спроектированная база данных содержит две основные сущности:

1. Фильм — основная сущность, предназначенная для хранения информации об избранных фильмах пользователя;
2. Служебная таблица базы данных, содержащая системные метаданные и хэши, используемые SQLite и Room для обеспечения целостности и управления структурой базы данных.

Сущность «Фильм» включает в себя следующие основные атрибуты:

- уникальный идентификатор фильма;
- название фильма;
- год выпуска;
- краткое описание;
- путь к постеру.

Данная структура является минимально достаточной для хранения информации об избранных фильмах и соответствует функциональным требованиям приложения.

Доступ к данным

Для взаимодействия с базой данных используется паттерн DAO (Data Access Object). Интерфейс MovieDao определяет набор операций для работы с сущностью «Фильм», включая:

- добавление фильма в базу данных
- удаление фильма из базы данных
- получение списка всех сохраненных фильмов

Класс MovieDatabase отвечает за создание и управление экземпляром базы данных, а также за предоставление доступа к DAO-интерфейсам. Взаимодействие с базой данных осуществляется исключительно через слой ViewModel, что соответствует принципам архитектуры MVVM и обеспечивает разделение ответственности между компонентами приложения.

Обработка данных

Обработка данных в приложении осуществляется на двух уровнях:

- удалённые данные, получаемые от сервера через Kinopoisk API, используются для отображения актуальной информации о фильмах, трейлерах и рецензиях;
- локальные данные, хранящиеся в базе данных, используются для формирования списка избранных фильмов и обеспечения офлайн-доступа.

Добавление фильма в избранное инициируется пользователем на экране детальной информации о фильме. После этого соответствующий объект сохраняется в локальной базе данных. Удаление фильма из избранного приводит к удалению записи из базы данных. Все операции с базой данных выполняются асинхронно, что предотвращает блокировку пользовательского интерфейса.

Использование локальной базы данных на основе Room является оптимальным решением для данного проекта. Оно позволяет обеспечить хранение пользовательских данных, поддержку офлайн-режима и удобную интеграцию с архитектурой MVVM без необходимости развёртывания и сопровождения серверной инфраструктуры.

Простота структуры базы данных, ограниченное количество сущностей и чёткое разделение ответственности между компонентами приложения делают систему хранения данных надёжной, понятной и легко расширяемой. В дальнейшем база данных может быть дополнена новыми сущностями, например, для хранения пользовательских настроек или истории просмотров, без существенных изменений архитектуры приложения.

3.3 Проектирование объектной модели приложения

Проектирование объектной модели является важным этапом разработки Android-приложения, так как позволяет формализовать структуру системы, определить основные классы, их обязанности и взаимосвязи. Объектная модель приложения описывает логическое представление программной системы

с точки зрения объектно-ориентированного подхода и служит основой для дальнейшей реализации и сопровождения приложения.

В рамках данного проекта объектная модель была спроектирована с использованием архитектурного паттерна MVVM (Model–View–ViewModel), что позволило чётко разделить пользовательский интерфейс, бизнес-логику и работу с данными. Для наглядного представления структуры системы была разработана UML-диаграмма классов, отражающая ключевые компоненты приложения и связи между ними.

Объектная модель приложения логически разделена на несколько слоёв:

- слой пользовательского интерфейса (UI layer);
- слой представления (ViewModel layer);
- слой работы с сетью (Network layer);
- слой хранения данных (Database layer);
- слой моделей данных (Models и DTO).

Такое разделение повышает читаемость кода, упрощает тестирование и обеспечивает гибкость при дальнейшем расширении функциональности приложения.

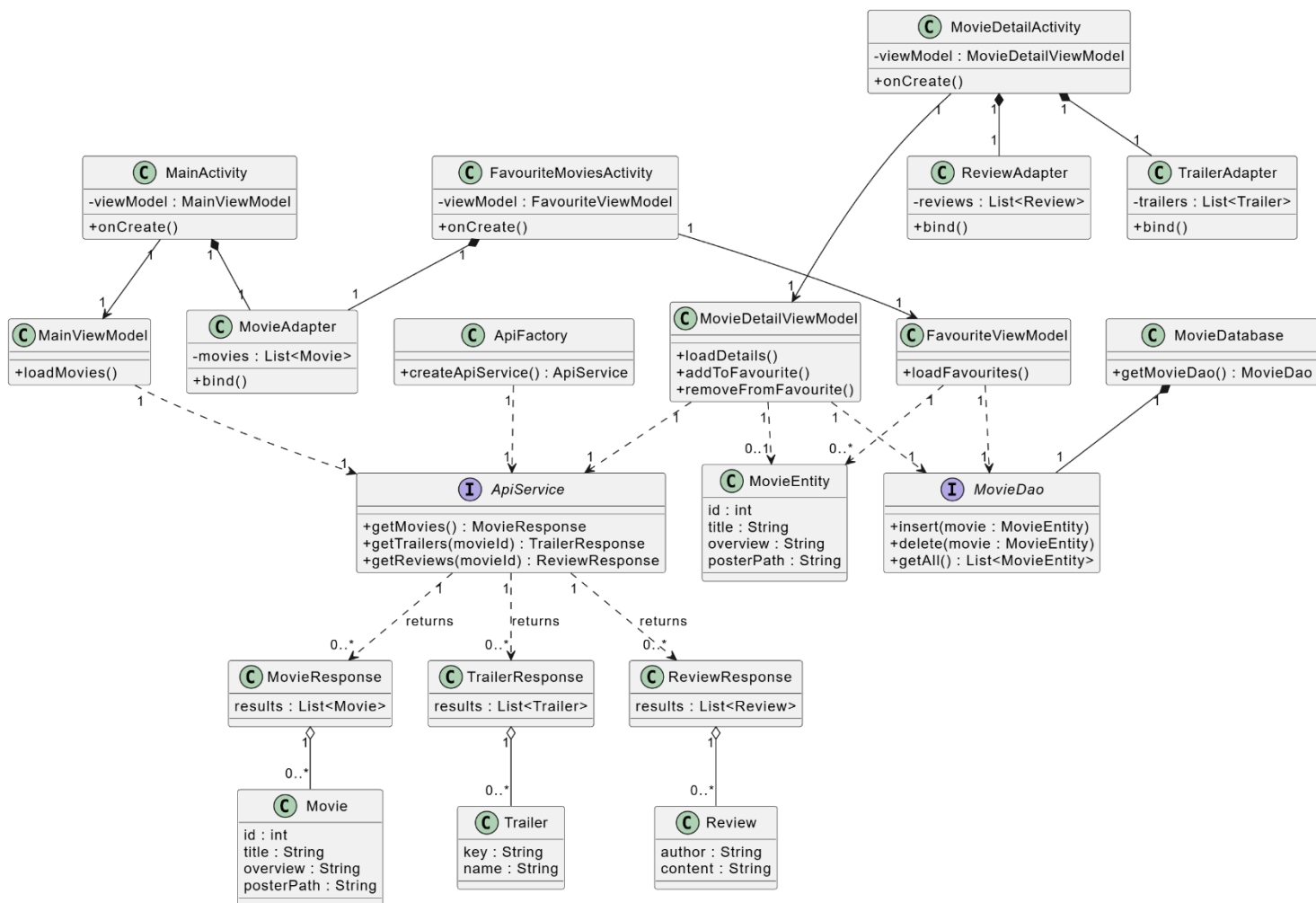


Рисунок 4 – Спроектированная объектная модель приложения.

Слой пользовательского интерфейса (UI layer)

Слой пользовательского интерфейса представлен классами MainActivity, MovieDetailActivity и FavouriteMoviesActivity. Эти классы отвечают за отображение экранов приложения и обработку пользовательских действий.

- MainActivity реализует главный экран приложения, на котором отображается список фильмов.

- `MovieDetailActivity` отвечает за экран детальной информации о выбранном фильме, включая трейлеры и отзывы.
- `FavouriteMoviesActivity` предназначен для отображения списка фильмов, добавленных пользователем в избранное.

Каждая `Activity` содержит ссылку на соответствующий `ViewModel`, через который осуществляется получение данных и выполнение операций. Такое взаимодействие исключает прямую работу UI с источниками данных и соответствует принципам MVVM.

Для отображения списков данных используются адаптеры `MovieAdapter`, `TrailerAdapter` и `ReviewAdapter`. Они инкапсулируют логику связывания данных моделей с элементами пользовательского интерфейса и используются в составе соответствующих `Activity`. Связь между `Activity` и адаптерами реализована как композиция, поскольку адаптеры создаются и используются исключительно в рамках соответствующих экранов.

Слой `ViewModel`

Слой `ViewModel` представлен классами `MainViewModel`, `MovieDetailViewModel` и `FavouriteViewModel`. Эти классы содержат бизнес-логику приложения и выступают связующим звеном между пользовательским интерфейсом и источниками данных.

- `MainViewModel` отвечает за загрузку списка фильмов с удалённого сервера.
- `MovieDetailViewModel` реализует получение детальной информации о фильме, а также операции добавления и удаления фильма из избранного.
- `FavouriteViewModel` отвечает за загрузку списка избранных фильмов из локальной базы данных.

`ViewModel`-классы взаимодействуют с сетевым API и базой данных через интерфейсы, что снижает связанность компонентов и упрощает модификацию реализации без изменения бизнес-логики.

Слой работы с сетью (Network layer)

Для взаимодействия с удалённым сервером используется интерфейс `ApiService`, который описывает методы получения данных о фильмах, трейлерах и отзывах. Возвращаемые данные представлены в виде объектов `MovieResponse`, `TrailerResponse` и `ReviewResponse`, содержащих соответствующие коллекции моделей.

Класс `ApiFactory` отвечает за создание экземпляра `ApiService` и инкапсулирует детали его инициализации. Такой подход позволяет централизовать настройку сетевого взаимодействия и повысить гибкость архитектуры.

Использование DTO-классов (Data Transfer Object) позволяет отделить сетевой формат данных от внутренних моделей приложения и упрощает обработку ответов сервера.

Слой хранения данных (Database layer)

Для локального хранения данных в приложении используется база данных, представленная классом `MovieDatabase`. Доступ к данным осуществляется через интерфейс `MovieDao`, реализующий операции добавления, удаления и получения фильмов.

Интерфейс `MovieDao` используется `ViewModel`-классами `MovieDetailViewModel` и `FavouriteViewModel`, что позволяет управлять избранными фильмами без прямого взаимодействия с механизмами хранения данных. Такой подход соответствует принципам инкапсуляции и разделения ответственности.

Сущность `MovieEntity` представляет собой объект, хранимый в базе данных, и содержит основные атрибуты фильма. Она используется исключительно на уровне хранения данных и не применяется напрямую в пользовательском интерфейсе.

Модели данных

Модели `Movie`, `Trailer` и `Review` используются для представления данных внутри приложения и отображения их в пользовательском интерфейсе.

Эти классы содержат только данные и не включают бизнес-логику, что соответствует принципу «анемичных моделей».

Связь между сетевыми DTO и моделями реализована через агрегацию: объекты `MovieResponse`, `TrailerResponse` и `ReviewResponse` содержат коллекции соответствующих моделей.

Выводы по объектной модели

Разработанная объектная модель приложения обеспечивает чёткое разделение ответственности между компонентами системы и соответствует современным рекомендациям по разработке Android-приложений. Использование паттерна MVVM, интерфейсов для доступа к данным и модульной структуры классов повышает поддерживаемость и расширяемость приложения.

UML-диаграмма классов наглядно демонстрирует структуру приложения, основные классы и характер их взаимодействия, что упрощает анализ архитектуры и дальнейшее развитие проекта.

3.4 Проектирование логики диалога с пользователем

Логику взаимодействия с пользователем в данном проекте было решено отобразить с помощью диаграммы состояний, представленной на Рисунке 5.

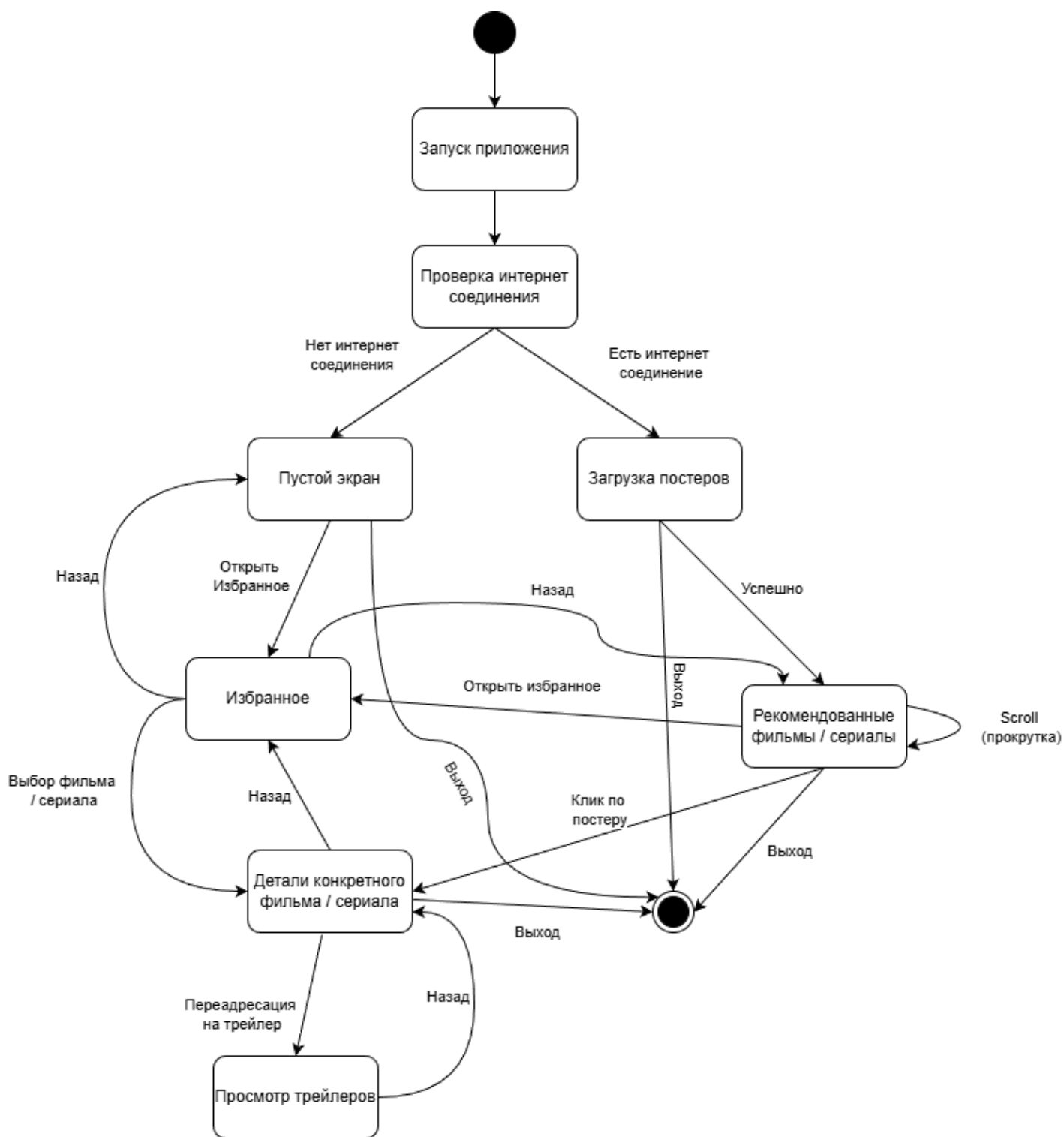


Рисунок 5 – Диаграмма состояний.

Диаграмма состояний позволяет наглядно представить возможные состояния приложения, события, инициируемые пользователем, а также реакции

системы на эти события. Такой подход упрощает анализ логики работы приложения и снижает вероятность логических ошибок на этапе реализации.

Логика диалога в приложении построена вокруг последовательного перехода пользователя между основными экранами: списком фильмов, экраном детальной информации о фильме и экраном избранных фильмов. Каждый экран соответствует отдельному состоянию системы, а действия пользователя (нажатия кнопок, выбор элементов списка) выступают в роли событий, вызывающих переходы между состояниями.

Начальным состоянием приложения является запуск главного экрана со списком фильмов. В этом состоянии пользователь может просматривать доступные фильмы, переходить к экрану избранного или выбирать конкретный фильм для просмотра подробной информации.

Главный экран

Данное состояние соответствует отображению списка фильмов, загруженных с удалённого сервера. Пользователь может прокручивать список, обновлять данные и выбирать интересующий фильм. Также с главного экрана возможен переход к экрану избранных фильмов.

Экран детальной информации

При выборе фильма пользователь переходит в состояние просмотра детальной информации. В этом состоянии отображаются основные сведения о фильме, а также связанные с ним трейлеры и отзывы. Пользователь может добавить фильм в избранное либо удалить его из списка избранных, после чего система обновляет состояние локального хранилища данных.

Экран «Избранное»

Данное состояние предназначено для просмотра списка фильмов, сохранённых пользователем в избранное. Пользователь может выбрать фильм для перехода к экрану детальной информации или вернуться на главный экран.

Переходы между состояниями

Переходы между состояниями инициируются действиями пользователя и сопровождаются соответствующими изменениями состояния приложения:

- выбор фильма в списке приводит к переходу с главного экрана или экрана избранного на экран детальной информации;
- нажатие кнопки возврата или элемента навигации возвращает пользователя на предыдущий экран;
- добавление фильма в избранное изменяет состояние данных, но не приводит к смене экрана;
- переход к экрану избранных осуществляется через элемент навигации с главного экрана.

Все переходы реализованы таким образом, чтобы пользователь всегда мог вернуться к предыдущему состоянию без потери контекста, что соответствует стандартным рекомендациям по проектированию мобильных интерфейсов.

Обработка пользовательских действий и устойчивость логики

Логика диалога спроектирована с учётом возможных повторных действий пользователя и изменения состояния данных. Например, повторное нажатие кнопки добавления фильма в избранное не приводит к дублированию записей в базе данных. Аналогично, при удалении фильма из избранного система корректно обновляет пользовательский интерфейс.

Асинхронные операции загрузки данных (получение списка фильмов, трейлеров и отзывов) не блокируют пользовательский интерфейс и не нарушают последовательность диалога. В случае временной недоступности данных приложение сохраняет текущее состояние экрана, что обеспечивает стабильность пользовательского опыта.

Выводы по логике диалога

Использование диаграммы состояний при проектировании логики диалога с пользователем позволило формализовать поведение приложения и чётко определить возможные сценарии взаимодействия. Такая модель делает

логику приложения прозрачной и понятной, упрощает реализацию навигации и снижает вероятность ошибок при обработке пользовательских действий.

Разработанная логика диалога обеспечивает интуитивно понятное взаимодействие с приложением, логичную навигацию между экранами и корректную реакцию системы на действия пользователя, что является важным фактором качества мобильного приложения.

4. РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА

Для разработки мобильного приложения для работы с фильмами была выбрана платформа Android, так как она является одной из наиболее распространённых мобильных операционных систем и предоставляет широкие возможности для создания интерактивных пользовательских интерфейсов и работы с сетевыми и локальными данными. В качестве языка программирования был использован Java, поскольку он является одним из основных языков разработки Android-приложений, обладает высокой стабильностью, развитой экосистемой библиотек и хорошо подходит для реализации многослойной архитектуры приложения.

Средой разработки проекта была выбрана Android Studio, которая является официальной интегрированной средой разработки для платформы Android и предоставляет полный набор инструментов для проектирования интерфейсов, написания кода, отладки и тестирования приложения. Использование Android Studio позволило эффективно организовать структуру проекта, управлять зависимостями и контролировать жизненный цикл компонентов приложения.

Для работы с удалёнными данными в приложении используется взаимодействие с внешним API, предоставляющим информацию о фильмах, трейлерах и рецензиях. Получение данных осуществляется по сети с использованием REST-подхода, что обеспечивает актуальность отображаемой информации и позволяет динамически обновлять контент. Для хранения пользовательских данных, таких как список избранных фильмов, применяется локальная база данных, что позволяет сохранять информацию между сессиями и обеспечивать доступ к избранному контенту без необходимости постоянного подключения к сети Интернет.

Архитектура приложения построена с разделением логики на уровни представления, обработки данных и работы с источниками данных. Такой подход повышает читаемость кода, упрощает его сопровождение и

масштабирование, а также способствует повторному использованию компонентов. В совокупности выбранные технологии и инструменты обеспечивают стабильную работу приложения, удобство взаимодействия с пользователем и соответствие современным требованиям к разработке мобильных приложений.

При запуске приложения изначально возможны 2 варианта – это успешно загруженные и отрисованные фильмы на стартовом экране, и пустой экран без загруженных фильмов.

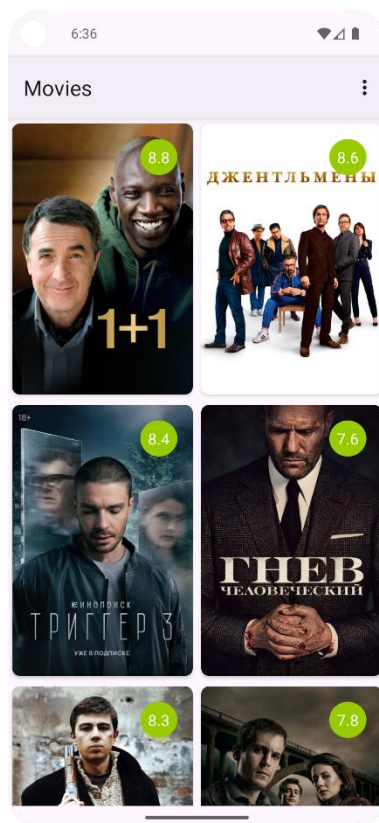


Рисунок 6 – Экран при успешной загрузке.

Соответственно также возможен вариант с пустой отрисовкой элементом. Такое может возникнуть, если на устройстве нет интернет-соединения, или на стороне сервера наблюдаются проблемы. Ниже на рисунке показан данный случай.



Рисунок 7 – Пустой экран при отсутствии интернет-соединения.

В случае успешной загрузки, пользователь может перейти по конкретному фильму / сериалу, кликнув по картинке на экране. В ответ на это действия у пользователя откроется новый экран с подробными деталями. Данный экран показан на рисунке ниже. Также пользователь на этом экране может добавить фильм в избранное, если он ему понравился, чтобы информация о нем сохранялась локально на телефоне, таким образом он не забудет о своих желаниях что ему посмотреть потом.

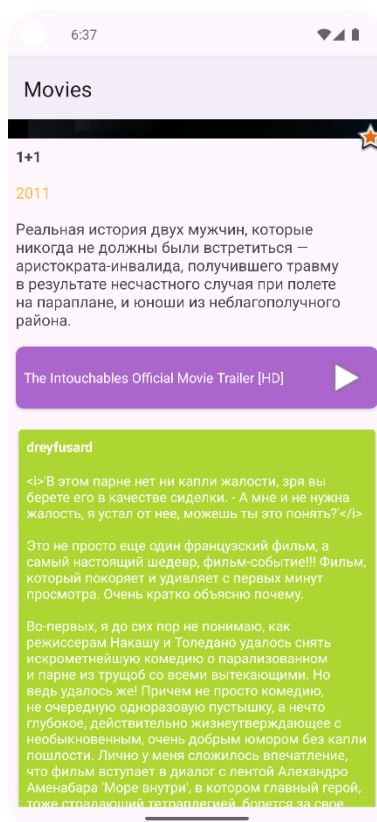


Рисунок 8 – Экран с подробными деталями.

Также пользователь может и перейти в раздел Избранное, где показыва-
ются все сохраненные фильмы. Здесь возможны 2 случая, если у пользователя
на устройстве есть интернет, то у него отображается все подаваемая информа-
ция в списке избранных. Если же у него нет интернета, то фильмы все равно
сохраняются, отображается постер, название, год и описание. Это говорит о
том, что несмотря ни на что, пользователь в любое время может зайти в свои
избранные, освежить информацию о фильмах и выбрать, что ему посмотреть
на этот раз. Рисунки с разделом Избранное представлены ниже.

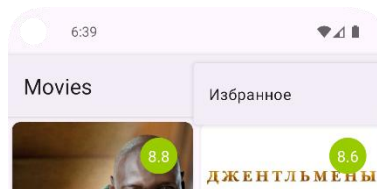


Рисунок 9 – Экран с баром Избранное.

Нажав на трей в баре и кнопку Избранное пользователь переходит в раз-
дел Избранное.

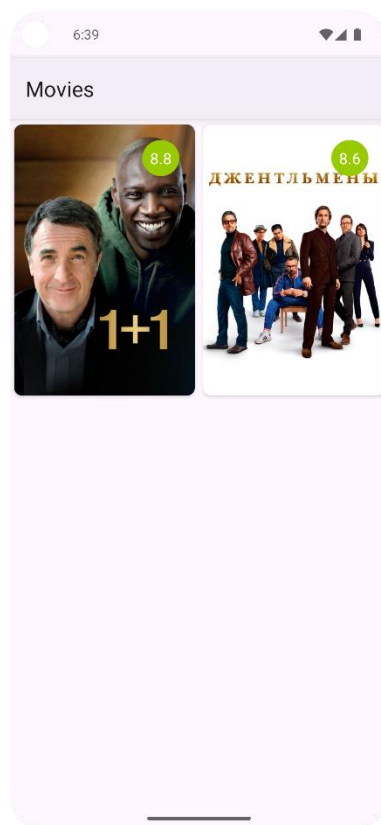


Рисунок 10 – Экран с избранными фильмами.

ЗАКЛЮЧЕНИЕ

В ходе разработки Android-приложения для работы с каталогом фильмов были охвачены основные этапы проектирования и реализации программных систем, включая анализ предметной области, проектирование архитектуры приложения, разработку объектной модели, а также организацию хранения и обработки данных. В процессе работы были использованы диаграммы вариантов использования, диаграмма классов и диаграмма состояний, что позволило формализовать требования и наглядно представить структуру и логику работы приложения.

Разработанное приложение предоставляет пользователю возможность просматривать список фильмов, получать подробную информацию о выбранных фильмах, включая описание, трейлеры и рецензии, а также формировать персональный список избранных фильмов. Использование внешнего API обеспечивает актуальность данных и позволяет динамически обновлять контент без необходимости ручного вмешательства.

Особое внимание в проекте было уделено архитектуре приложения и разделению ответственности между компонентами. Применение многоуровневой архитектуры с выделением слоёв пользовательского интерфейса, бизнес-логики и работы с данными повысило модульность системы, упростило сопровождение кода и обеспечило возможность дальнейшего расширения функциональности. Использование локальной базы данных позволило сохранять пользовательские предпочтения и обеспечивать доступ к избранным фильмам даже при отсутствии подключения к сети Интернет.

Заключительным этапом проекта стала интеграция пользовательского интерфейса и логики приложения, а также проверка корректности взаимодействия между экранами и источниками данных. Реализованная навигация и управление состояниями приложения обеспечивают понятный и удобный пользовательский опыт при работе с каталогом фильмов.

Разработанное Android-приложение является функциональным прототипом информационной системы для просмотра и управления фильмами, объединяющим сетевое взаимодействие, локальное хранение данных и современный подход к архитектуре мобильных приложений. Полученные в ходе выполнения проекта результаты демонстрируют практическое применение теоретических знаний в области объектно-ориентированного проектирования, мобильной разработки и архитектуры программных систем, а также создают основу для дальнейшего развития приложения, включая расширение функционала, оптимизацию производительности и улучшение пользовательского интерфейса.

СПИСОК ЛИТЕРАТУРЫ

1. Бурн, Э., Гриффитс, Д., Марсикано, К. Разработка приложений для Android / Э. Бурн, Д. Гриффитс, К. Марсикано; пер. с англ. – СПб.: Питер, 2023. – 1104 с.
2. Филлипс, Б., Харди, Б., Марсикано, К. Android. Программирование для профессионалов / Б. Филлипс, Б. Харди, К. Марсикано; пер. с англ. – 4-е изд. – СПб.: Питер, 2021. – 1104 с. – (Библиотека программиста).
3. Дейтел, П., Дейтел, Х., Уолд, А. Android для разработчиков / П. Дейтел, Х. Дейтел, А. Уолд; пер. с англ. – 3-е изд. – М.: Эксмо, 2021. – 912 с.
4. Голд, Н. Kotlin в действии / Н. Голд, С. Жемеров, А. Исаева; пер. с англ. – СПб.: Питер, 2022. – 496 с.
5. Сильеро, Д. Архитектура Android-приложений. Подход Clean Architecture на Kotlin и Java / Д. Сильеро; пер. с англ. – М.: ДМК Пресс, 2023. – 428 с.

ПРИЛОЖЕНИЕ А

Листинг файла ApiFactory.java

```
package com.example.movies;

import retrofit2.Retrofit;
import retrofit2.adapter.rxjava3.RxJava3CallAdapterFactory;
import retrofit2.converter.gson.GsonConverterFactory;

public class ApiFactory {

    private static final String BASE_URL = "https://api.kinopoisk.dev/v1.4/";

    private static final Retrofit retrofit = new Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .addCallAdapterFactory(RxJava3CallAdapterFactory.create())
        .build();

    public static final ApiService apiService = retrofit.create(ApiService.class);
}
```

Листинг файла ApiService.java

```
package com.example.movies;

import io.reactivex.rxjava3.core.Single;
import retrofit2.http.GET;
import retrofit2.http.Headers;
import retrofit2.http.Path;
import retrofit2.http.Query;

public interface ApiService {

    @GET("movie?token=S582NST-8P3434Q-NRRMRVD-HZWGV2M&field=rating.kp&search=4-6&sort-Field=votes.kp&sortType=-1&limit=100")
    Single<MovieResponse> loadMovies(@Query("page") int page);

    @GET("movie/{idFilms}?token=S582NST-8P3434Q-NRRMRVD-HZWGV2M")
    Single<TrailerResponse> loadTrailers(@Path("idFilms") int idFilms);

    @GET("review?page=1&limit=10&selectFields=&sortField=createdAt&sortType=1")
    @Headers("X-API-KEY: S582NST-8P3434Q-NRRMRVD-HZWGV2M")
    Single<ReviewResponse> loadReviews(@Query("movieId") int id);
}
```

Листинг файла FavouriteMoviesActivity.java

```
package com.example.movies;

import android.content.Context;
import android.content.Intent;
import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.lifecycle.Observer;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

public class FavouriteMoviesActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {

```

```

super.onCreate(savedInstanceState);
EdgeToEdge.enable(this);
setContentView(R.layout.activity_favourite_movies);
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
    Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
    v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
    return insets;
});

RecyclerView recyclerViewMovies = findViewById(R.id.recyclerViewMovies);
MoviesAdapter moviesAdapter = new MoviesAdapter();
recyclerViewMovies.setLayoutManager(new GridLayoutManager(this, 2));
recyclerViewMovies.setAdapter(moviesAdapter);
moviesAdapter.setOnMovieClickListener(new MoviesAdapter.OnMovieClickListener() {
    @Override
    public void onMovieClick(Movie movie) {
        Intent intent = MovieDetailActivity.newIntent(
            FavouriteMoviesActivity.this,
            movie
        );
        startActivity(intent);
    }
});

FavouriteMoviesViewModel viewModel = new ViewModelProvider(this).get(
    FavouriteMoviesViewModel.class
);
viewModel.getMovies().observe(this, new Observer<List<Movie>>() {
    @Override
    public void onChanged(List<Movie> movies) {
        moviesAdapter.setMovies(movies);
    }
});
}

public static Intent newIntent(Context context){
    return new Intent(context, FavouriteMoviesActivity.class);
}
}

```

Листинг файла FavouriteMoviesViewModel.java

```

package com.example.movies;

import android.app.Application;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;

import java.util.List;

public class FavouriteMoviesViewModel extends AndroidViewModel {

    private MovieDao movieDao;
    public FavouriteMoviesViewModel(@NonNull Application application) {
        super(application);
        movieDao = MovieDatabase.getInstance(application).movieDao();
    }

    public LiveData<List<Movie>> getMovies(){
        return movieDao.getAllFavouriteMovies();
    }
}

```

Листинг файла MainActivity.java

```

package com.example.movies;

import android.content.Intent;

```

```

import android.os.Bundle;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.ProgressBar;

import androidx.activity.EdgeToEdge;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.lifecycle.Observer;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.GridLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import java.util.List;

import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
import io.reactivex.rxjava3.core.Scheduler;
import io.reactivex.rxjava3.functions.Consumer;
import io.reactivex.rxjava3.schedulers.Schedulers;

public class MainActivity extends AppCompatActivity {
    private MainViewModel viewModel;

    private RecyclerView recyclerViewMovies;
    private ProgressBar progressBarLoading;
    private MoviesAdapter moviesAdapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

        progressBarLoading = findViewById(R.id.progressBarLoading);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        recyclerViewMovies = findViewById(R.id.recyclerViewMovies);
        moviesAdapter = new MoviesAdapter();
        recyclerViewMovies.setAdapter(moviesAdapter);
        recyclerViewMovies.setLayoutManager(new GridLayoutManager(this, 2));
        viewModel = new ViewModelProvider(this).get(MainViewModel.class);
        viewModel.getMovies().observe(this, new Observer<List<Movie>>() {
            @Override
            public void onChanged(List<Movie> movies) {
                moviesAdapter.setMovies(movies);
            }
        });
        viewModel.loadMovies();
        viewModel.getIsLoading().observe(this, new Observer<Boolean>() {
            @Override
            public void onChanged(Boolean isLoading) {
                if (isLoading) {
                    progressBarLoading.setVisibility(View.VISIBLE);
                } else {
                    progressBarLoading.setVisibility(View.GONE);
                }
            }
        });
    }
}

```

```

        moviesAdapter.setOnReachEndListener(new MoviesAdapter.OnReachEndListener() {
            @Override
            public void OnReachEnd() {
                viewModel.loadMovies();
            }
        });

        moviesAdapter.setOnMovieClickListener(new MoviesAdapter.OnMovieClickListener() {
            @Override
            public void onMovieClick(Movie movie) {
                Intent intent = MovieDetailActivity.newIntent(MainActivity.this, movie);
                startActivity(intent);
            }
        });
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main_menu, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(@NonNull MenuItem item) {
        if (item.getItemId() == R.id.itemFavourite){
            Intent intent = FavouriteMoviesActivity.newIntent(this);
            startActivity(intent);
        }
        return super.onOptionsItemSelected(item);
    }
}

```

Листинг файла MainViewModel.java

```

package com.example.movies;

import android.app.Application;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;

import java.util.List;

import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
import io.reactivex.rxjava3.core.Scheduler;
import io.reactivex.rxjava3.disposables.CompositeDisposable;
import io.reactivex.rxjava3.disposables.Disposable;
import io.reactivex.rxjava3.functions.Action;
import io.reactivex.rxjava3.functions.Consumer;
import io.reactivex.rxjava3.schedulers.Schedulers;

public class MainViewModel extends AndroidViewModel {

    private static final String TAG = "MainViewModel";

    // нужно подписываться на коллекцию фильмов в activity, для этого создаем данный объект
    private final MutableLiveData<List<Movie>> movies = new MutableLiveData<>();
    private final MutableLiveData<Boolean> isLoading = new MutableLiveData<>(false);

    private final CompositeDisposable compositeDisposable = new CompositeDisposable();

    private int page = 1;

    public MainViewModel(@NonNull Application application) {

```

```

        super(application);
        loadMovies();
    }

    public LiveData<List<Movie>> getMovies() {
        return movies;
    }

    public LiveData<Boolean> getIsLoading() {
        return isLoading;
    }

    // метод по загрузке данных
    public void loadMovies(){
        Boolean loading = isLoading.getValue();
        if (loading != null && loading){
            return;
        }
        Disposable disposable = ApiFactory.apiService.loadMovies(page)
            .subscribeOn(Schedulers.io())
            .observeOn(AndroidSchedulers.mainThread())
            .doOnSubscribe(new Consumer<Disposable>() {
                @Override
                public void accept(Disposable disposable) throws Throwable {
                    isLoading.setValue(true);
                }
            })
            .doAfterTerminate(new Action() {
                @Override
                public void run() throws Throwable {
                    isLoading.setValue(false);
                }
            })
            .subscribe(new Consumer<MovieResponse>() {
                @Override
                public void accept(MovieResponse movieResponse) throws Throwable {
                    List<Movie> loadedMovies = movies.getValue();
                    if (loadedMovies != null){
                        loadedMovies.addAll(movieResponse.getMovies());
                        movies.setValue(loadedMovies);
                    } else {
                        // нужно установить прилетевшие значения из response в livedata
                        movies.setValue(movieResponse.getMovies());
                    }
                    page++;
                }
            }, new Consumer<Throwable>() {
                @Override
                public void accept(Throwable throwable) throws Throwable {
                    Log.d(TAG, throwable.toString());
                }
            });
        compositeDisposable.add(disposable);
    }

    @Override
    protected void onCleared() {
        super.onCleared();
        compositeDisposable.dispose();
    }
}

```

Листинг файла Movie.java

```

package com.example.movies;

import androidx.room.Embedded;
import androidx.room.Entity;
import androidx.room.PrimaryKey;

```

```

import com.google.gson.annotations.SerializedName;

import java.io.Serializable;
@Entity(tableName = "favourite_movies")
public class Movie implements Serializable {

    @PrimaryKey
    @SerializedName("id")
    private int id;
    @SerializedName("name")
    private String name;
    @SerializedName("description")
    private String description;
    @SerializedName("year")
    private int year;
    @SerializedName("poster")
    @Embedded
    private Poster poster;
    @SerializedName("rating")
    @Embedded
    private Rating rating;

    public Movie(int id, String name, String description, int year, Poster poster, Rating rating) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.year = year;
        this.poster = poster;
        this.rating = rating;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public int getYear() {
        return year;
    }

    public Poster getPoster() {
        return poster;
    }

    public Rating getRating() {
        return rating;
    }

    @Override
    public String toString() {
        return "Movie{" +
            "id=" + id +
            ", name=" + name + "\" +
            ", description=" + description + "\" +
            ", year=" + year +
            ", poster=" + poster +
            ", rating=" + rating +
            '\'';
    }
}

```

```
}
```

Листинг файла MovieDao.java

```
package com.example.movies;

import androidx.lifecycle.LiveData;
import androidx.room.Dao;
import androidx.room.Insert;
import androidx.room.Query;

import java.util.List;

import io.reactivex.rxjava3.core.Completable;

@Dao
public interface MovieDao {

    @Query("SELECT * FROM favourite_movies")
    LiveData<List<Movie>> getAllFavouriteMovies();

    @Query("SELECT * FROM favourite_movies WHERE id = :movieId")
    LiveData<Movie> getFavouriteMovie(int movieId);

    @Insert
    Completable insertMovie(Movie movie);

    @Query("DELETE FROM favourite_movies WHERE id = :movieId")
    Completable removeMovie (int movieId);
}
```

Листинг файла MovieDatabase.java

```
package com.example.movies;

import android.app.Application;

import androidx.room.Database;
import androidx.room.Room;
import androidx.room.RoomDatabase;
@Database(entities = {Movie.class}, version = 1, exportSchema = false)
public abstract class MovieDatabase extends RoomDatabase {

    private static final String DB_NAME = "movie.db";
    private static MovieDatabase instance = null;

    public static MovieDatabase getInstance(Application application){
        if (instance == null){
            instance = Room.databaseBuilder(
                application,
                MovieDatabase.class,
                DB_NAME
            ).build();
        }
        return instance;
    }

    abstract MovieDao movieDao();
}
```

Листинг файла MovieDetailActivity.java

```
package com.example.movies;

import android.content.Context;
import android.content.Intent;
import android.graphics.drawable.Drawable;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.ImageView;
```

```

import android.widget.TextView;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.lifecycle.Observer;
import androidx.lifecycle.ViewModelProvider;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.bumptech.glide.Glide;

import java.util.List;

import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
import io.reactivex.rxjava3.core.Scheduler;
import io.reactivex.rxjava3.functions.Consumer;
import io.reactivex.rxjava3.schedulers.Schedulers;

public class MovieDetailActivity extends AppCompatActivity {

    private static final String TAG = "MovieDetailActivity";

    private static final String EXTRA_MOVIE = "movie";

    private MovieDetailViewModel viewModel;

    private ImageView imageViewPoster;
    private TextView textViewTitle;
    private TextView textViewYear;
    private TextView textViewDescription;
    private RecyclerView recyclerViewTrailers;
    private RecyclerView recyclerViewReviews;
    private ReviewsAdapter reviewsAdapter;
    private TrailersAdapter trailersAdapter;
    private ImageView imageViewStar;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_movie_detail);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        viewModel = new ViewModelProvider(this).get(MovieDetailViewModel.class);
        initView();
        trailersAdapter = new TrailersAdapter();
        reviewsAdapter = new ReviewsAdapter();
        recyclerViewTrailers.setAdapter(trailersAdapter);
        recyclerViewReviews.setAdapter(reviewsAdapter);

        Movie movie = (Movie) getIntent().getSerializableExtra(EXTRA_MOVIE);

        Glide.with(this)
            .load(movie.getPoster().getUrl())
            .into(imageViewPoster);
        textViewTitle.setText(movie.getName());
        textViewYear.setText(String.valueOf(movie.getYear()));
    }

```

```

textViewDescription.setText(movie.getDescription());

viewModel.loadTrailers(movie.getId());
viewModel.getTrailers().observe(this, new Observer<List<Trailer>>() {
    @Override
    public void onChanged(List<Trailer> trailers) {
        trailersAdapter.setTrailers(trailers);
    }
});
trailersAdapter.setOnTrailerClickListener(new TrailersAdapter.OnTrailerClickListener() {
    @Override
    public void onTrailerClick(Trailer trailer) {
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse(trailer.getUrl()));
        intent.setPackage("com.android.chrome");
        startActivity(intent);
    }
});
viewModel.getReviews().observe(this, new Observer<List<Review>>() {
    @Override
    public void onChanged(List<Review> reviewList) {
        reviewsAdapter.setReviews(reviewList);
    }
});
viewModel.loadReviews(movie.getId());

Drawable starOff = ContextCompat.getDrawable(
    MovieDetailActivity.this,
    android.R.drawable.star_big_off);
Drawable starOn = ContextCompat.getDrawable(
    MovieDetailActivity.this,
    android.R.drawable.star_big_on);
viewModel.getFavouriteMovie(movie.getId()).observe(this, new Observer<Movie>() {
    @Override
    public void onChanged(Movie movieFromDb) {
        if (movieFromDb == null){
            imageViewStar.setImageDrawable(starOff);
            imageViewStar.setOnClickListener(new View.OnClickListener() {
                @Override
                public void onClick(View v) {
                    viewModel.insertMovie(movie);
                }
            });
        } else {
            imageViewStar.setImageDrawable(starOn);
            imageViewStar.setOnClickListener(new View.OnClickListener() {
                @Override
                public void onClick(View v) {
                    viewModel.removeMovie(movie.getId());
                }
            });
        }
    }
});

private void initView(){
    imageViewPoster = findViewById(R.id.imageViewPoster);
    textViewTitle = findViewById(R.id.textViewTitle);
    textViewYear = findViewById(R.id.textViewYear);
    textViewDescription = findViewById(R.id.textViewDescription);
    recyclerViewTrailers = findViewById(R.id.recyclerViewTrailers);
    recyclerViewReviews = findViewById(R.id.recyclerViewReviews);
    imageViewStar = findViewById(R.id.imageViewStar);
}

public static Intent newIntent(Context context, Movie movie){

```

```

        Intent intent = new Intent(context, MovieDetailActivity.class);
        intent.putExtra(EXTRA_MOVIE, movie);
        return intent;
    }
}

```

Листинг файла MovieDetailViewModel.java

```

package com.example.movies;

import android.app.Application;
import android.util.Log;

import androidx.annotation.NonNull;
import androidx.lifecycle.AndroidViewModel;
import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;

import java.util.Collections;
import java.util.List;

import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
import io.reactivex.rxjava3.disposables.CompositeDisposable;
import io.reactivex.rxjava3.disposables.Disposable;
import io.reactivex.rxjava3.functions.Consumer;
import io.reactivex.rxjava3.functions.Function;
import io.reactivex.rxjava3.schedulers.Schedulers;

public class MovieDetailViewModel extends AndroidViewModel {

    private static final String TAG = "MovieDetailViewModel";

    private final CompositeDisposable compositeDisposable = new CompositeDisposable();
    private final MutableLiveData<List<Trailer>> trailers = new MutableLiveData<>();
    private final MutableLiveData<List<Review>> reviews = new MutableLiveData<>();

    private final MovieDao movieDao;

    public MovieDetailViewModel(@NonNull Application application) {
        super(application);
        movieDao = MovieDatabase.getInstance(application).movieDao();
    }

    public LiveData<Movie> getFavouriteMovie(int movieId){
        return movieDao.getFavouriteMovie(movieId);
    }

    public LiveData<List<Trailer>> getTrailers() {
        return trailers;
    }

    public LiveData<List<Review>> getReviews() {
        return reviews;
    }

    public void loadReviews(int id){
        Disposable disposable = ApiFactory.apiService.loadReviews(id)
            .subscribeOn(Schedulers.io())
            .observeOn(AndroidSchedulers.mainThread())
            .map(new Function<ReviewResponse, List<Review>>() {
                @Override
                public List<Review> apply(ReviewResponse reviewResponse) throws Throwable {
                    return reviewResponse.getReviews();
                }
            })
            .subscribe(new Consumer<List<Review>>() {
                @Override
                public void accept(List<Review> reviewList ) throws Throwable {
                    reviews.setValue(reviewList);
                }
            });
    }
}

```

```

        }
        }, new Consumer<Throwable>() {
            @Override
            public void accept(Throwable throwable) throws Throwable {
                Log.d(TAG, throwable.toString());
            }
        });
        compositeDisposable.add(disposable);
    }

    public void insertMovie(Movie movie){
        Disposable disposable = movieDao.insertMovie(movie)
            .subscribeOn(Schedulers.io())
            .subscribe();
        compositeDisposable.add(disposable);
    }

    public void removeMovie(int movieId){
        Disposable disposable = movieDao.removeMovie(movieId)
            .subscribeOn(Schedulers.io())
            .subscribe();
        compositeDisposable.add(disposable);
    }

    public void loadTrailers (int id){
        Disposable disposable = ApiFactory.apiService.loadTrailers(id)
            .subscribeOn(Schedulers.io())
            .observeOn(AndroidSchedulers.mainThread())
            .map(new Function<TrailerResponse, List<Trailer>>() {
                @Override
                public List<Trailer> apply(TrailerResponse trailerResponse) throws Throwable {
                    return trailerResponse.getTrailersList().getTrailers();
                }
            })
            .subscribe(new Consumer<List<Trailer>>() {
                @Override
                public void accept(List<Trailer> trailerList) throws Throwable {
                    trailers.setValue(trailerList);
                }
            }, new Consumer<Throwable>() {
                @Override
                public void accept(Throwable throwable) throws Throwable {
                    Log.d(TAG, throwable.toString());
                }
            });
        compositeDisposable.add(disposable);
    }

    @Override
    protected void onCleared() {
        super.onCleared();
        compositeDisposable.dispose();
    }
}

```

Листинг файла MovieResponse.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

import java.util.List;

public class MovieResponse {

    // коллекция фильмов из запроса api
    @SerializedName("docs")
    private List<Movie> movies;
}

```

```

    public MovieResponse(List<Movie> movies) {
        this.movies = movies;
    }

    public List<Movie> getMovies() {
        return movies;
    }

    @Override
    public String toString() {
        return "MovieResponse{" +
            "movies=" + movies +
            '}';
    }
}

```

Листинг файла MoviesAdapter.java

```

package com.example.movies;

// адаптер нужен для RecyclerView чтобы отображать элементы на экране

import android.graphics.drawable.Drawable;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.RecyclerView;

import com.bumptech.glide.Glide;

import java.util.ArrayList;
import java.util.List;

public class MoviesAdapter extends RecyclerView.Adapter<MoviesAdapter.MovieViewHolder>{

    // внутри адаптера нужно передавать коллекцию фильмов
    private List<Movie> movies = new ArrayList<>();
    private OnReachEndListener onReachEndListener;
    private OnMovieClickListener onMovieClickListener;

    // из активности мы можем вызывать данный сеттер, переопределять
    // слушатель и определять поведение при достижении конца списка
    public void setOnReachEndListener(OnReachEndListener onReachEndListener) {
        this.onReachEndListener = onReachEndListener;
    }

    public void setMovies(List<Movie> movies) {
        this.movies = movies;
        notifyDataSetChanged();
    }

    public void setOnMovieClickListener(OnMovieClickListener onMovieClickListener) {
        this.onMovieClickListener = onMovieClickListener;
    }

    @NonNull
    @Override
    public MovieViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        // нужно из макета создать View, затем из этой View создать экземпляр класса ViewHolder
        View view = LayoutInflater.from(parent.getContext()).inflate(
            R.layout.movie_item,
            parent,
            false
        );
    }
}

```

```

    );

    return new MovieViewHolder(view);
}

@Override
public void onBindViewHolder(@NonNull MovieViewHolder holder, int position) {
    Log.d("MoviesAdapter", "onBindViewHolder: " + position);
    Movie movie = movies.get(position);
    // необходимо установить картинку в ImageView, делаем это при помощи Glide
    if(movie.getPoster()!=null) {
        Glide.with(holder.itemView)
            .load(movie.getPoster().getUrl())
            .into(holder.imageViewPoster);
    } else if (movie.getPoster() == null) {
        Glide.with(holder.itemView)
            .load("https://st.kp.yandex.net/images/no-poster.gif")
            .into(holder.imageViewPoster);
    }

    double rating = movie.getRating().getKp();
    int backgroundId;
    if (rating > 7){
        backgroundId = R.drawable.circle_green;
    } else if (rating > 5) {
        backgroundId = R.drawable.circle_orange;
    } else {
        backgroundId = R.drawable.circle_red;
    }
    Drawable background = ContextCompat.getDrawable(holder.itemView.getContext(), backgroundId);
    holder.textViewRating.setBackground(background);
    holder.textViewRating.setText(String.valueOf(rating).substring(0,3));

    if (position > movies.size() - 10 && onReachEndListener != null){
        onReachEndListener.OnReachEnd();
    }

    holder.itemView.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            if (onMovieClickListener != null){
                onMovieClickListener.onMovieClick(movie);
            }
        }
    });
}

@Override
public int getItemCount() {
    return movies.size();
}

interface OnReachEndListener{

    void OnReachEnd();
}

interface OnMovieClickListener{

    void onMovieClick(Movie movie);
}

static class MovieViewHolder extends RecyclerView.ViewHolder{

    private final ImageView imageViewPoster;
    private final TextView textViewRating;

```

```

        public MovieViewHolder(@NonNull View itemView) {
            super(itemView);
            imageViewPoster = itemView.findViewById(R.id.imageViewPoster);
            textViewRating = itemView.findViewById(R.id.textViewRating);
        }
    }
}

```

Листинг файла Poster.java

```

package com.example.movies;

import androidx.annotation.NonNull;

import com.google.gson.annotations.SerializedName;

import java.io.Serializable;

public class Poster implements Serializable {

    @SerializedName("url")
    private String url;

    public Poster(String url) {
        this.url = url;
    }

    public String getUrl() {
        return url;
    }

    @NonNull
    @Override
    public String toString() {
        return "Poster{" +
            "url=\"" + url + "\" +
            '}'";
    }
}

```

Листинг файла Rating.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

import java.io.Serializable;

public class Rating implements Serializable {

    @SerializedName("kp")
    private double kp;

    public Rating(double kp) {
        this.kp = kp;
    }

    public double getKp() {
        return kp;
    }

    @Override
    public String toString() {
        return "Rating{" +
            "kp=\"" + kp + "\" +
            '}'";
    }
}

```

Листинг файла Reviewting.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

public class Review {

    @SerializedName("author")
    private String author;
    @SerializedName("review")
    private String review;
    @SerializedName("type")
    private String type;

    public Review(String author, String review, String type) {
        this.author = author;
        this.review = review;
        this.type = type;
    }

    public String getAuthor() {
        return author;
    }

    public String getReview() {
        return review;
    }

    public String getType() {
        return type;
    }

    @Override
    public String toString() {
        return "Review{" +
            "author=" + author + "\" +
            ", review=" + review + "\" +
            ", type=" + type + "\" +
            '}'";
    }
}

```

Листинг файла ReviewResponse.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

import java.util.List;

public class ReviewResponse {

    @SerializedName("docs")
    private List<Review> reviews;

    public ReviewResponse(List<Review> reviews) {
        this.reviews = reviews;
    }

    public List<Review> getReviews() {
        return reviews;
    }

    @Override
    public String toString() {
        return "ReviewResponse{" +
            "reviews=" + reviews +
            '}'";
    }
}

```

```
}  
}
```

Листинг файла ReviewsAdapter.java

```
package com.example.movies;  
  
import android.view.LayoutInflater;  
import android.view.View;  
import android.view.ViewGroup;  
import android.widget.LinearLayout;  
import android.widget.TextView;  
  
import androidx.annotation.NonNull;  
import androidx.core.content.ContextCompat;  
import androidx.recyclerview.widget.RecyclerView;  
  
import java.util.ArrayList;  
import java.util.List;  
  
public class ReviewsAdapter extends RecyclerView.Adapter<ReviewsAdapter.ReviewViewHolder>{  
  
    private static final String TYPE_NEGATIVE = "Негативный";  
    private static final String TYPE_POSITIVE = "Позитивный";  
    private static final String TYPE_NEUTRAL = "Нейтральный";  
  
    private List<Review> reviews = new ArrayList<>();  
  
    public void setReviews(List<Review> reviews) {  
        this.reviews = reviews;  
        notifyDataSetChanged();  
    }  
  
    @NonNull  
    @Override  
    public ReviewViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {  
        View view = LayoutInflater.from(parent.getContext()).inflate(  
            R.layout.review_item,  
            parent,  
            false  
        );  
        return new ReviewViewHolder(view);  
    }  
  
    @Override  
    public void onBindViewHolder(@NonNull ReviewViewHolder holder, int position) {  
        Review review = reviews.get(position);  
        holder.textViewAuthor.setText(review.getAuthor());  
        holder.textViewReview.setText(review.getReview());  
  
        String type = review.getType();  
        int colorResId = android.R.color.holo_red_light;  
        switch (type){  
            case TYPE_POSITIVE:  
                colorResId = android.R.color.holo_green_light;  
                break;  
            case TYPE_NEUTRAL:  
                colorResId = android.R.color.holo_orange_light;  
                break;  
        }  
        int color = ContextCompat.getColor(holder.itemView.getContext(), colorResId);  
        holder.linearLayoutContainer.setBackgroundColor(color);  
    }  
  
    @Override  
    public int getItemCount() {  
        return reviews.size();  
    }  
}
```

```

static class ReviewViewHolder extends RecyclerView.ViewHolder {

    private final LinearLayout linearLayoutContainer;
    private final TextView textViewAuthor;
    private final TextView textViewReview;

    public ReviewViewHolder(@NonNull View itemView) {
        super(itemView);
        linearLayoutContainer = itemView.findViewById(R.id.linearLayoutContainer);
        textViewAuthor = itemView.findViewById(R.id.textViewAuthor);
        textViewReview = itemView.findViewById(R.id.textViewReview);
    }
}
}

```

Листинг файла Trailer.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

public class Trailer {

    @SerializedName("name")
    private String name;
    @SerializedName("url")
    private String url;

    public Trailer(String name, String url) {
        this.name = name;
        this.url = url;
    }

    public String getName() {
        return name;
    }

    public String getUrl() {
        return url;
    }

    @Override
    public String toString() {
        return "Trailer{" +
            "name=" + name + "\" +
            ", url=" + url + "\" +
            '}'";
    }
}

```

Листинг файла TrailerResponse.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;

public class TrailerResponse {

    @SerializedName("videos")
    private TrailersList trailersList;

    public TrailerResponse(TrailersList trailersList) {
        this.trailersList = trailersList;
    }

    public TrailersList getTrailersList() {
        return trailersList;
    }
}

```

```

@Override
public String toString() {
    return "TrailerResponse{" +
        "trailersList=" + trailersList +
        '}';
}
}

```

Листинг файла TrailersAdapter.java

```

package com.example.movies;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;
import java.util.List;

public class TrailersAdapter extends RecyclerView.Adapter<TrailersAdapter.TrailersViewHolder> {

    private List<Trailer> trailers = new ArrayList<>();
    private OnTrailerClickListener onTrailerClickListener;

    public void setTrailers(List<Trailer> trailers) {
        this.trailers = trailers;
        notifyDataSetChanged();
    }

    public void setOnTrailerClickListener(OnTrailerClickListener onTrailerClickListener) {
        this.onTrailerClickListener = onTrailerClickListener;
    }

    @NonNull
    @Override
    public TrailersViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext()).inflate(
            R.layout.trailer_item, parent, false);
        return new TrailersViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull TrailersViewHolder holder, int position) {
        Trailer trailer = trailers.get(position);
        holder.textViewTrailerName.setText(trailer.getName());
        holder.itemView.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (onTrailerClickListener != null){
                    onTrailerClickListener.onTrailerClick(trailer);
                }
            }
        });
    }

    @Override
    public int getItemCount() {
        return trailers.size();
    }

    interface OnTrailerClickListener {

        void onTrailerClick(Trailer trailer);
    }
}

```

```

static class TrailersViewHolder extends RecyclerView.ViewHolder{

    private final TextView textViewTrailerName;

    public TrailersViewHolder(@NonNull View itemView) {
        super(itemView);
        textViewTrailerName = itemView.findViewById(R.id.textViewTrailerName);
    }
}
}

```

Листинг файла TrailersList.java

```

package com.example.movies;

import com.google.gson.annotations.SerializedName;
import java.util.List;

public class TrailersList {

    @SerializedName("trailers")
    private List<Trailer> trailers;

    public TrailersList(List<Trailer> trailers) {
        this.trailers = trailers;
    }

    public List<Trailer> getTrailers() {
        return trailers;
    }

    @Override
    public String toString() {
        return "TrailersList{" +
            "trailers=" + trailers +
            '}';
    }
}

```

ПРИЛОЖЕНИЕ Б

Листинг файла activity_favourite_movies.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FavouriteMoviesActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerViewMovies"
        android:layout_width="0dp"

```

```

        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Листинг файла activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerViewMovies"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <ProgressBar
        android:id="@+id/progressBarLoading"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:visibility="gone"
        tools:visibility="visible"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintBottom_toBottomOf="parent"/>

</androidx.constraintlayout.widget.ConstraintLayout>

```

Листинг файла activity_movie_detail.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MovieDetailActivity">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <ImageView
            android:id="@+id/imageViewPoster"
            android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:contentDescription="@null"
        app:layout_constraintTop_toTopOf="parent"
        android:adjustViewBounds="true"
        tools:src="@tools:sample/avatars"
    />

    <TextView
        android:id="@+id/textViewTtitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@+id/imageViewPoster"
        android:textSize="16sp"
        android:padding="8dp"
        android:textStyle="bold"
        tools:text="Title"
    />

    <TextView
        android:id="@+id/textViewYear"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@+id/textViewTtitle"
        android:textSize="16sp"
        android:padding="8dp"
        android:textColor="@android:color/holo_orange_light"
        tools:text="1990"
    />

    <TextView
        android:id="@+id/textViewDescription"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="8dp"
        android:textSize="16sp"
        app:layout_constraintTop_toBottomOf="@+id/textViewYear"
        tools:text="Description of movie" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerViewTrailers"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
        android:nestedScrollingEnabled="false"
        app:layout_constraintTop_toBottomOf="@+id/textViewDescription"
        tools:listitem="@layout/trailer_item" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recyclerViewReviews"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
        android:nestedScrollingEnabled="false"
        app:layout_constraintTop_toBottomOf="@+id/recyclerViewTrailers"
    />

```

```

<ImageView
    android:id="@+id/imageViewStar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:barrierMargin="16dp"
    tools:srcCompat="@android:drawable/btn_star_big_on"
    app:layout_constraintBottom_toBottomOf="@id/imageViewPoster"
    app:layout_constraintTop_toBottomOf="@id/imageViewPoster"
    app:layout_constraintEnd_toEndOf="@id/imageViewPoster"
/>

</androidx.constraintlayout.widget.ConstraintLayout>

</androidx.core.widget.NestedScrollView>

```

Листинг файла movie_item.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:cardCornerRadius="8dp"
    app:cardUseCompatPadding="true">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <ImageView
            android:minHeight="250dp"
            android:id="@+id/imageViewPoster"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:adjustViewBounds="true"
            android:contentDescription="@null"
            app:layout_constraintTop_toTopOf="parent"
            tools:src="@tools:sample/avatars" />

        <TextView
            android:id="@+id/textViewRating"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_margin="16dp"
            tools:background="@drawable/circle_green"
            android:textColor="@android:color/white"
            android:textSize="16sp"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            tools:text="8.0" />

    </androidx.constraintlayout.widget.ConstraintLayout>

```

```
</androidx.cardview.widget.CardView>
```

Листинг файла review_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    app:cardUseCompatPadding="true"
    android:layout_margin="8dp"
    app:cardCornerRadius="4dp"
    xmlns:tools="http://schemas.android.com/tools">

    <LinearLayout
        android:id="@+id/linearLayoutContainer"
        android:layout_width="match_parent"
        android:orientation="vertical"
        android:alpha="0.8"
        tools:background="@android:color/holo_red_light"
        android:layout_height="wrap_content">

        <TextView
            android:id="@+id/textViewAuthor"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/white"
            android:textStyle="bold"
            android:layout_margin="8dp"
            tools:text="Author"
            />

        <TextView
            android:id="@+id/textViewReview"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            tools:text="Very long text"
            android:textColor="@color/white"
            android:layout_margin="8dp"
            />

    </LinearLayout>
</androidx.cardview.widget.CardView>
```

Листинг файла trailer_item.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="4dp"
    app:cardBackgroundColor="@android:color/holo_purple">
```

```

app:cardCornerRadius="8dp"
app:cardUseCompatPadding="true">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:orientation="horizontal"
    android:padding="8dp">

    <TextView
        android:id="@+id/textViewTrailerName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textColor="@color/white"
        android:layout_gravity="center"
        tools:text="Трейлер" />

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="48dp"
        android:layout_height="48dp"
        app:srcCompat="@android:drawable/ic_media_play" />

</LinearLayout>

</androidx.cardview.widget.CardView>

```